

Trabajo de GitHub

Nombre

Sergio Andrés López Sepúlveda

Instructor

Luis Fernando Sánchez Pérez

Ficha

3169892

Servicio Nacional de Aprendizaje SENA

Centro De Tecnología De La Manufactura Avanzada CTMA

Análisis y Desarrollo De Software

2025

Tabla de Contenido

Introducción	4
Preguntas de Introducción:.....	5
¿Qué es un repositorio en Git y cuál es su función dentro de un proyecto de desarrollo?	5
¿Qué problema resuelve el uso de sistemas de control de versiones como Git en proyectos de desarrollo de software?	5
¿Cómo permite Git la colaboración entre varios desarrolladores sin que sus cambios interfieran entre sí?	5
¿Cuál es la diferencia entre un commit y un push en el flujo de trabajo de Git?	6
¿Qué es un pull request en GitHub y cómo ayuda a revisar y fusionar los cambios de los colaboradores?	6
¿Qué ventajas ofrece el uso de ramas (branches) en Git cuando se está desarrollando una nueva funcionalidad o corrigiendo un error?	6
¿En qué situaciones se recomienda usar un fork en GitHub?.....	7
¿Cómo podría estructurar el flujo de trabajo del equipo utilizando Git y GitHub para asegurar que todos los miembros colaboren de manera eficiente y no se presenten conflictos de código?	7
Imagine que un miembro del equipo ha realizado cambios importantes en una funcionalidad de la evaluación de la rueda de la vida, pero al hacer un merge de la rama en la que estaba trabajando, se presentan conflictos con la rama principal del proyecto. ¿Cómo resolvería este conflicto utilizando Git y GitHub?	8
Como parte de su rol en el proyecto, decide automatizar las pruebas unitarias de las funcionalidades de la web utilizando GitHub Actions. ¿Qué pasos seguiría para configurar un	

flujo de trabajo en GitHub Actions que ejecute las pruebas automáticamente con cada push a la rama principal y despliegue el sitio web de manera continua?	8
Actividad 1: Configuración Inicial	8
Evidencias:.....	9
Actividad 2: Ramas.....	11
Evidencias:.....	12
Actividad 3: Pull Request e Issues.....	15
Evidencias:.....	15
Actividad 4: Actions	19
Evidencias:.....	19
Conclusiones	22
Referencias Bibliográficas	23

Introducción

GitHub es una forma fácil y simple de administrar, compartir y mejorar el código de diferentes aplicaciones y programas. Tiene la posibilidad de la creación de un repositorio donde puedes tener el código que necesites administrar con funciones como las ramas que permiten tener un control sobre los cambios que se le añadirán, los issues para tener un orden de los problemas en el código y arreglarlos, los GitHub Actions que logran automatizar procesos y demás funciones útiles que se aprenderán en la realización de este trabajo.

Preguntas de Introducción:

¿Qué es un repositorio en Git y cuál es su función dentro de un proyecto de desarrollo?

Un repositorio es el elemento más básico de GitHub. Es un lugar donde puedes almacenar el código, los archivos y el historial de revisiones de cada archivo. Los repositorios pueden contar con múltiples colaboradores y pueden ser públicos como privados.

¿Qué problema resuelve el uso de sistemas de control de versiones como Git en proyectos de desarrollo de software?

Los sistemas de control de versiones (SCV) desempeñan un papel fundamental en la gestión de proyectos de software, ya que permiten registrar y controlar los cambios realizados en el código fuente a lo largo del tiempo. La importancia de los SCV radica en su capacidad para rastrear todas las modificaciones, facilitar la colaboración entre desarrolladores, y garantizar la integridad y la disponibilidad del código en todo momento.

Al emplear un SCV, los equipos de desarrollo pueden coordinar sus esfuerzos de manera eficiente, reducir conflictos en la integración de código, y revertir a versiones anteriores en caso de errores o problemas inesperados. Esto se traduce en un flujo de trabajo más organizado, una mayor visibilidad de los cambios realizados, y una mayor confiabilidad en el producto final.

¿Cómo permite Git la colaboración entre varios desarrolladores sin que sus cambios interfieran entre sí?

Una de las prácticas más importantes en el trabajo colaborativo con Git es crear una nueva rama para cada característica o tarea específica. Esto les permite a los desarrolladores trabajar en paralelo sin interferir en el trabajo de otros. Cada rama representa una unidad lógica de trabajo y puede fusionarse en la rama principal cuando la característica esté completa y probada.

¿Cuál es la diferencia entre un commit y un push en el flujo de trabajo de Git?

La diferencia básica entre git commit y git push es que el alcance de git commit es el repositorio local, y el de git push es el repositorio remoto.

El comando git push siempre viene después de ejecutar el comando git commit.

Cuando ejecutamos un comando git commit, se captura una instantánea de los cambios realizados actualmente en el proyecto. El comando git add realiza la puesta en escena de los cambios.

El comando git push empuja el contenido del repositorio local a un repositorio remoto. Este comando transfiere los commits del repositorio local al repositorio remoto.

¿Qué es un pull request en GitHub y cómo ayuda a revisar y fusionar los cambios de los colaboradores?

Un Pull Request (PR) es una solicitud para fusionar cambios desde una rama a otra. Permite a otros miembros del equipo revisar, discutir y aprobar los cambios antes de fusionarlos. Los PRs son esenciales para mantener la calidad del código y garantizar que todos los cambios sean revisados antes de integrarlos en la rama principal.

¿Qué ventajas ofrece el uso de ramas (branches) en Git cuando se está desarrollando una nueva funcionalidad o corrigiendo un error?

Trabajar con ramas en Git es muy útil, ya que no solo te permite mantener el orden en tu código, sino que también facilita la colaboración entre varios desarrolladores y, por tanto, aumenta la productividad del equipo. Por eso, quiero contarte cuáles son sus principales ventajas:

1. Puedes aislar los cambios: Al crear diferentes ramas, puedes trabajar en nuevas funcionalidades o correcciones sin afectar el código principal.

2. Colabora con otros desarrolladores: Cada desarrollador puede trabajar en su propia rama, lo que es muy útil porque previene conflictos.
3. Controla el historial: Las ramas permiten organizar mejor el historial de commits, haciéndolo más claro y fácil de seguir.
4. Realiza pruebas seguras: Puedes experimentar con ideas sin miedo a dañar el código en producción.

¿En qué situaciones se recomienda usar un fork en GitHub?

Si clonas un repositorio que era tuyo, podrás realizar cambios en local y subirlos a GitHub siempre que quieras. Pero si el repositorio era de otro desarrollador y tú no tenías permisos de escritura sobre él, entonces no podrás subir cambios, porque GitHub no te lo permitirá. Para este caso es donde se necesita un fork.

Entonces un fork es una copia de un repositorio, pero creado en tu propia cuenta de GitHub, donde sí que tienes permisos de escritura. Por tanto, si tienes intención de bajarte un repositorio de GitHub para hacer cambios en él y ese repositorio no te pertenece, lo más normal es que crees un fork primero y luego clones en local tu propio fork.

¿Cómo podría estructurar el flujo de trabajo del equipo utilizando Git y GitHub para asegurar que todos los miembros colaboren de manera eficiente y no se presenten conflictos de código?

Utilizando ramas y verificaciones con GitHub actions que permitan que cada uno trabaje en la parte que le corresponde sin interferir con el otro y no se suban archivos inválidos sin antes verificar que estos si funcionen con nombres claros de que se ha cambiado y para qué.

Imagine que un miembro del equipo ha realizado cambios importantes en una funcionalidad de la evaluación de la rueda de la vida, pero al hacer un merge de la rama en la que estaba trabajando, se presentan conflictos con la rama principal del proyecto.

¿Cómo resolvería este conflicto utilizando Git y GitHub?

Se debe revisar que archivos tienen conflicto, Git los marcara de una forma entendible en la que uno elije que datos se van a quedar o reemplazar editándolos manualmente, luego se hace un git add archivo, git commit y por último git push a la rama principal para comprobar que el conflicto se ha arreglado.

Como parte de su rol en el proyecto, decide automatizar las pruebas unitarias de las funcionalidades de la web utilizando GitHub Actions. ¿Qué pasos seguiría para configurar un flujo de trabajo en GitHub Actions que ejecute las pruebas automáticamente con cada push a la rama principal y despliegue el sitio web de manera continua?

Se debe crear un archivo .yml en la ruta de .github/workflows/ en tu repositorio que será usado por GitHub como un Workflow, este va a contener unas instrucciones a seguir, y se tendrá que especificar en el archivo que se debe activar al hacer push a una rama elegida y usando si se prefiere los GitHub Actions del Marketplace de GitHub para automatizar el proceso sin intervención externa y poder verificar que todo sea correcto fácilmente.

Actividad 1: Configuración Inicial

1. Crear su perfil de usuario en la plataforma GitHub (<https://www.github.com>).
2. Crear en su perfil una organización (Ej: Coquito Amarillo S.A.S)
3. Crear en su organización un proyecto (Ej: Web Corporativa)
4. Crear en su proyecto un repositorio con un nombre apropiado, como “coquito amarillo-web”.

5. Clonar el repositorio en su máquina local utilizando ``git clone ``.
6. Crear la estructura inicial de directorios dentro del repositorio (por ejemplo, una carpeta ``documentación`` para la gestión administrativa del proyecto, ``src`` para los archivos del código y una carpeta ``assets`` para imágenes, estilos y otros recursos).
7. Realizar el primer commit para guardar la estructura general y los archivos base del proyecto (Readme, Licenciamiento, Requerimientos, product backlog, HTML, CSS, imágenes).
8. Subir los cambios al repositorio remoto con el comando ``git push``.

Evidencias:

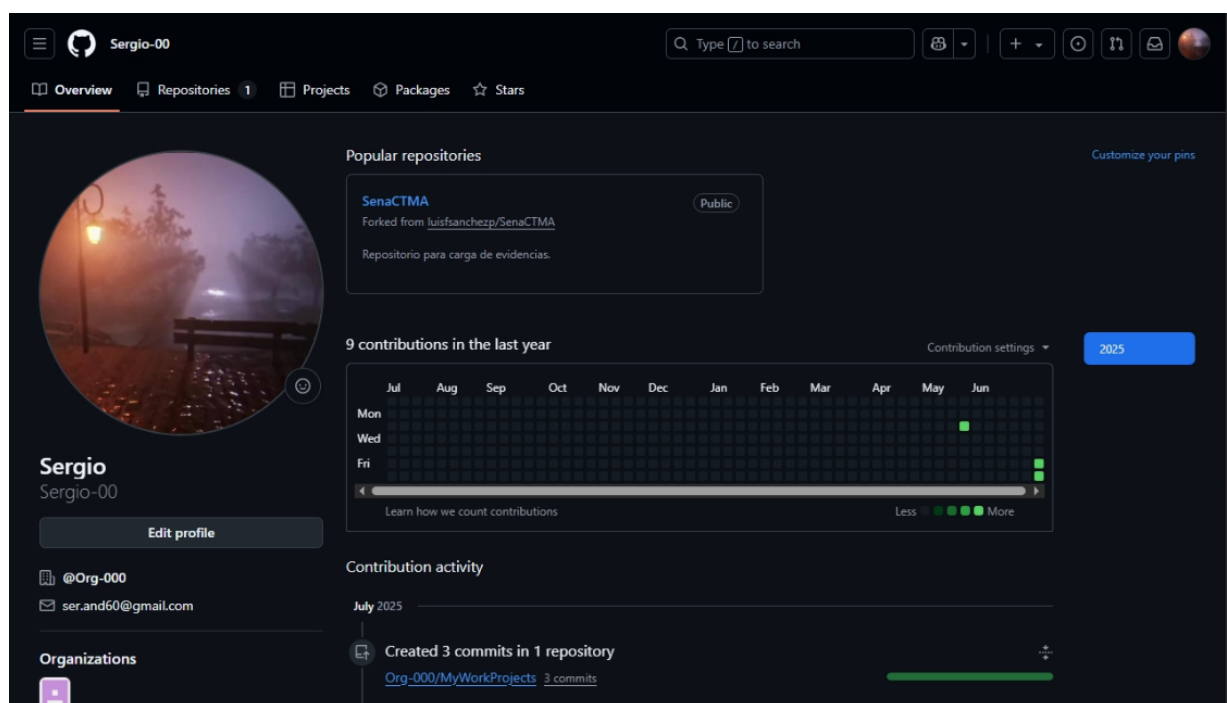


Ilustración 1. GitHub - Mi perfil

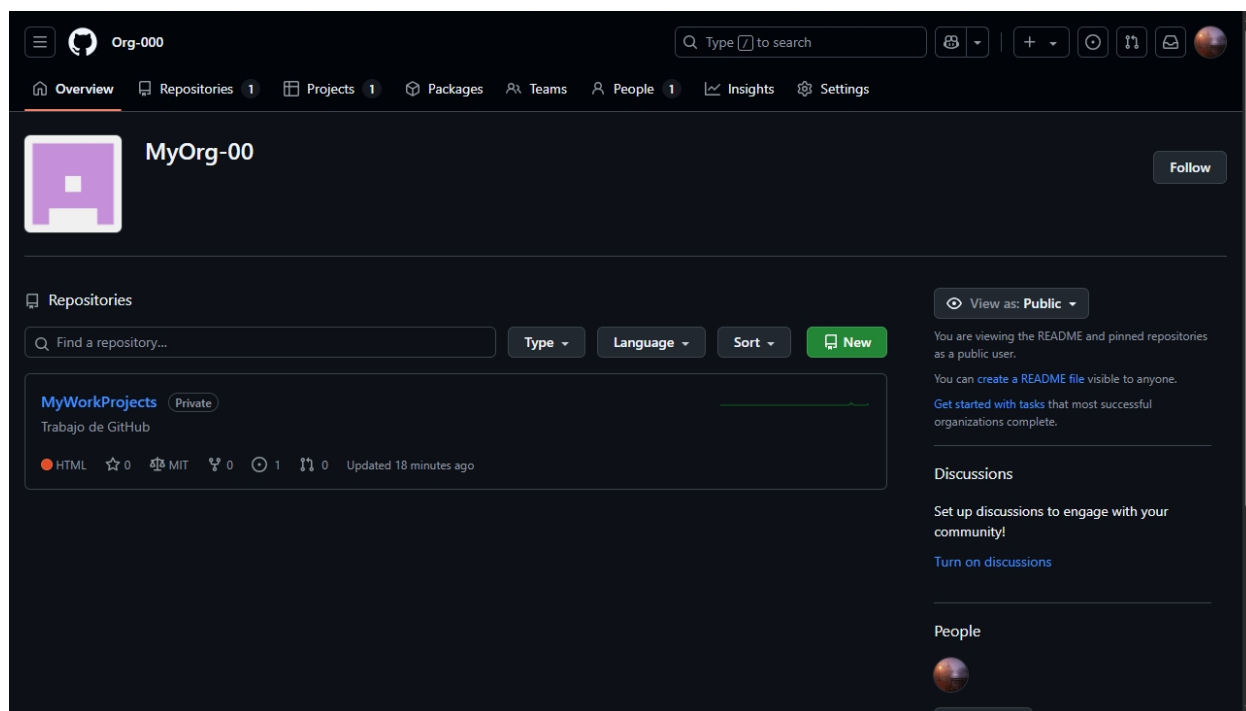


Ilustración 2. GitHub - Mi organización

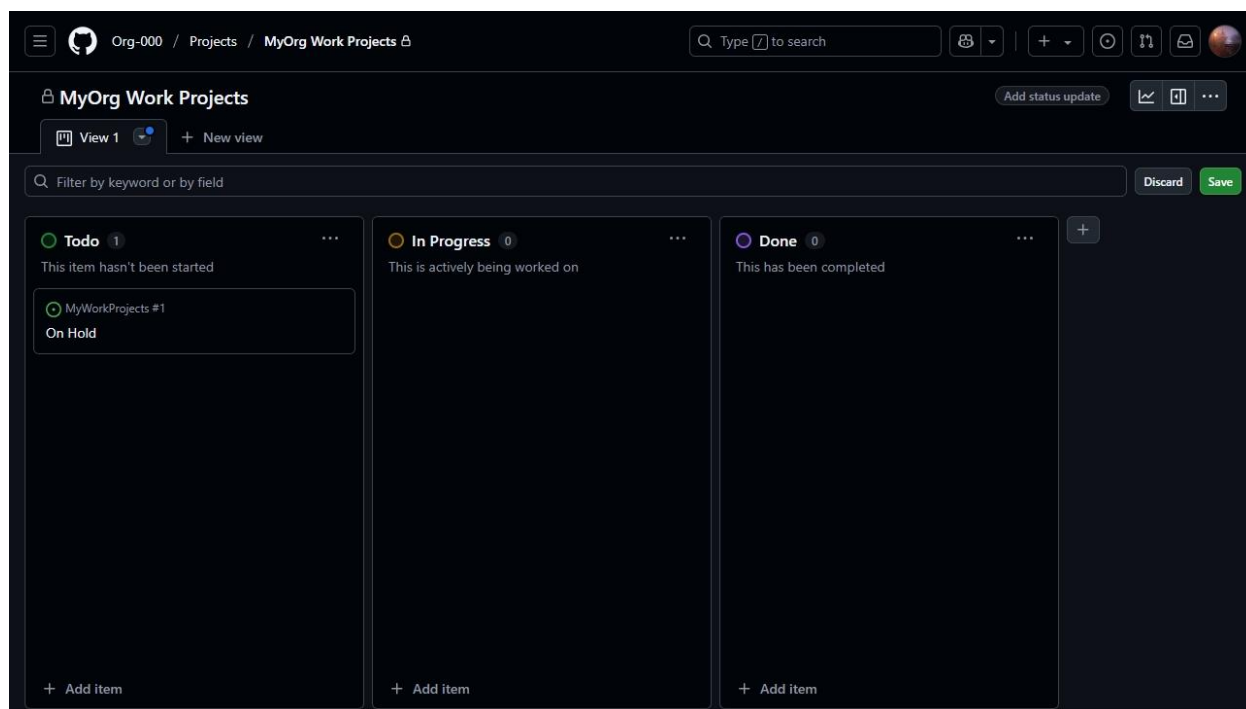


Ilustración 3. GitHub – El proyecto de mi organización con el repositorio

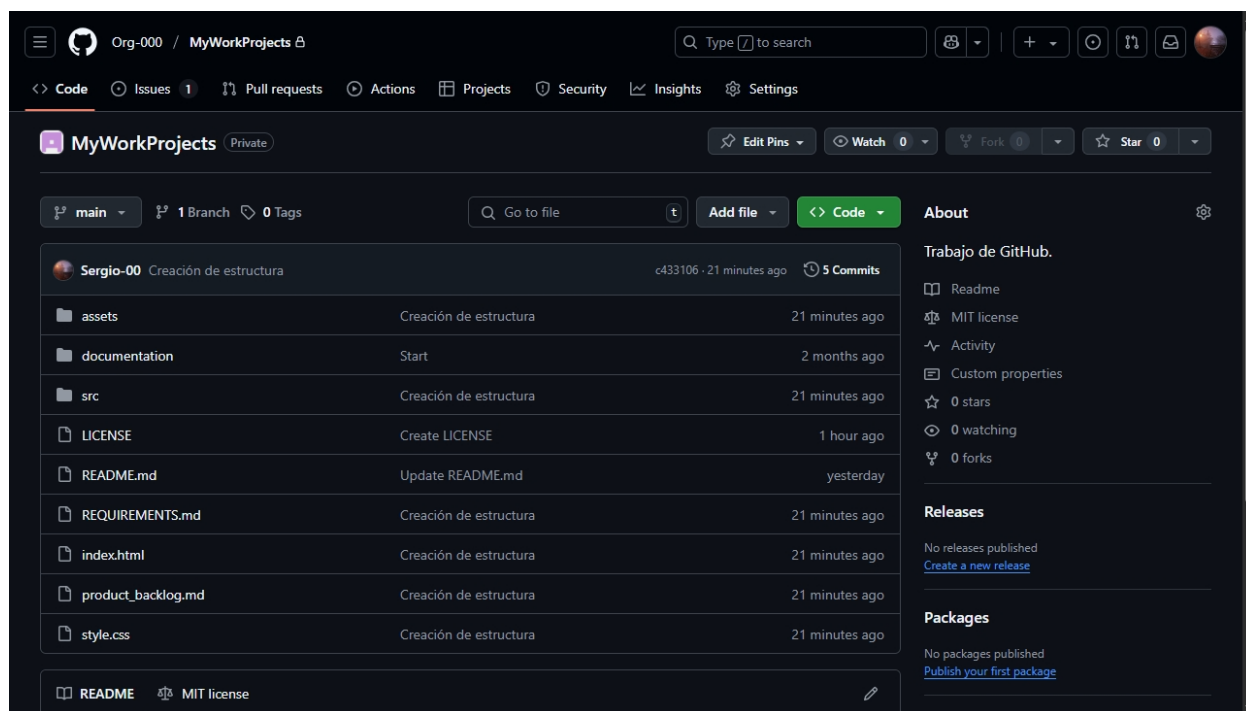
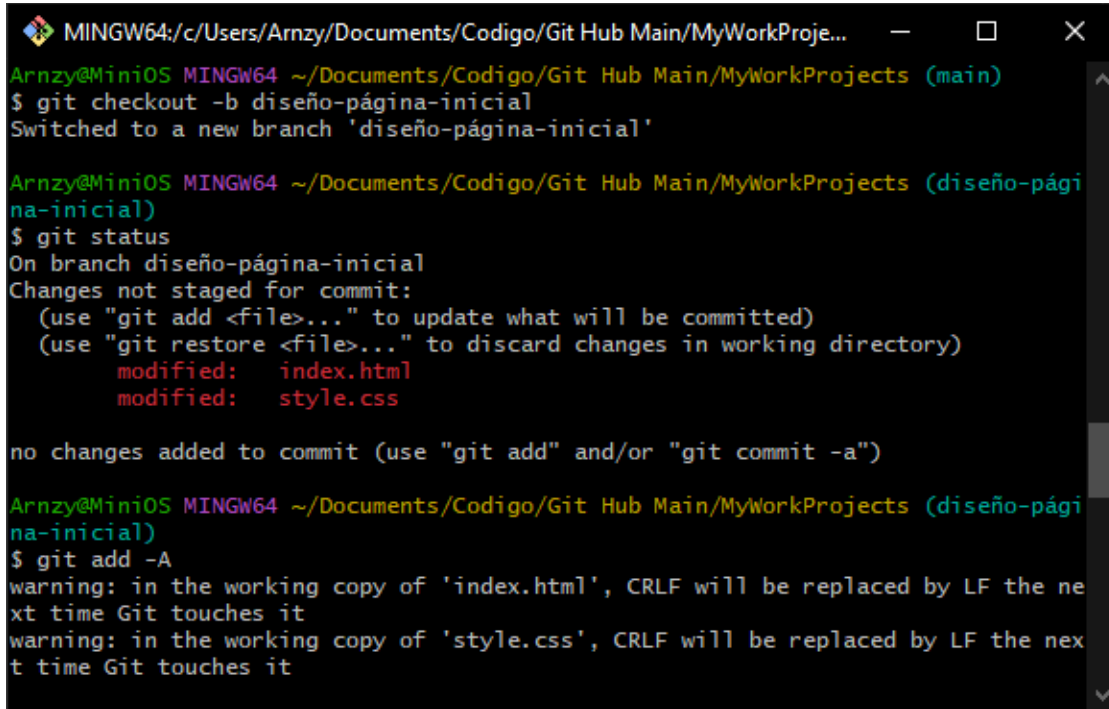


Ilustración 4. GitHub – El repositorio de la organización después del git push

Actividad 2: Ramas

1. Crear una nueva rama para una funcionalidad específica, como el diseño de la página principal o la implementación de la evaluación de la rueda de la vida. a. Ejemplo: ``git checkout -b diseño-página-inicial``
2. Trabajar en los archivos correspondientes (HTML, CSS, Python) dentro de esa rama.
3. Realizar commits frecuentemente para registrar los avances realizados en la rama.
4. Una vez completada la funcionalidad, realizar un merge de la rama con la rama principal ('main') mediante un pull request en GitHub.
5. Resolver cualquier conflicto que pueda surgir al intentar fusionar las ramas.

Evidencias:



```

MINGW64:/c/Users/Arnzy/Documents/Codigo/Git Hub Main/MyWorkProje...
Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (main)
$ git checkout -b diseño-página-inicial
Switched to a new branch 'diseño-página-inicial'

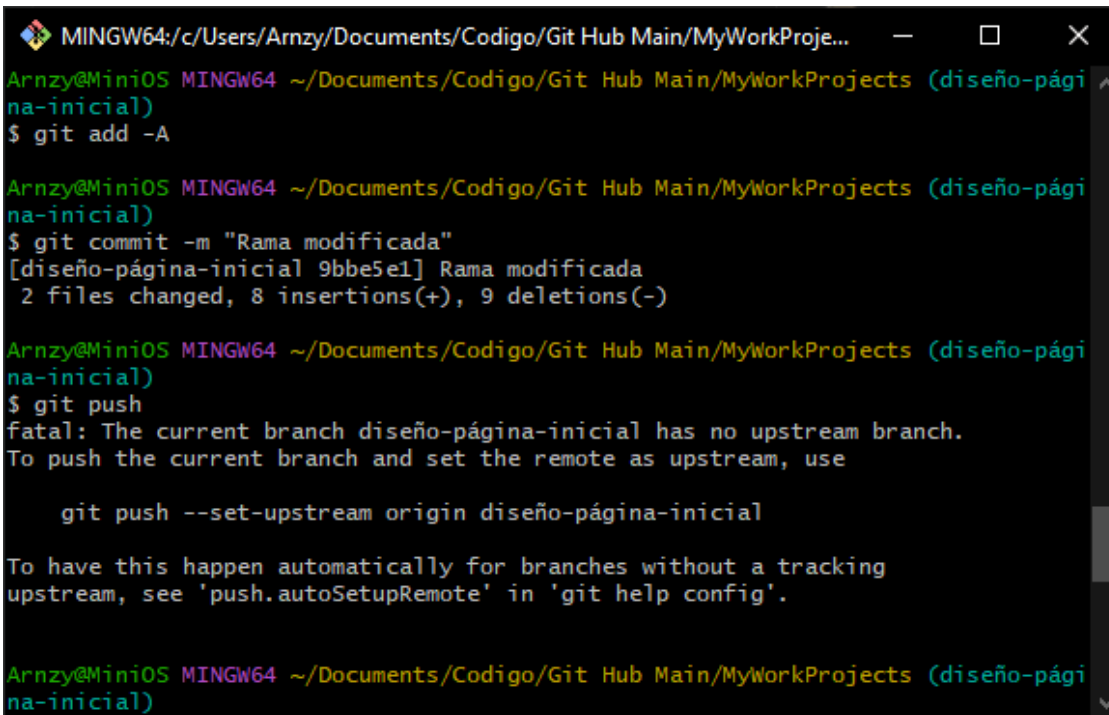
Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (diseño-página-inicial)
$ git status
On branch diseño-página-inicial
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html
        modified:   style.css

no changes added to commit (use "git add" and/or "git commit -a")

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (diseño-página-inicial)
$ git add -A
warning: in the working copy of 'index.html', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'style.css', CRLF will be replaced by LF the next time Git touches it

```

Ilustración 5. Git Bash - Creación de la Rama



```

MINGW64:/c/Users/Arnzy/Documents/Codigo/Git Hub Main/MyWorkProje...
Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (diseño-página-inicial)
$ git add -A

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (diseño-página-inicial)
$ git commit -m "Rama modificada"
[diseño-página-inicial 9bbe5e1] Rama modificada
2 files changed, 8 insertions(+), 9 deletions(-)

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (diseño-página-inicial)
$ git push
fatal: The current branch diseño-página-inicial has no upstream branch.
To push the current branch and set the remote as upstream, use

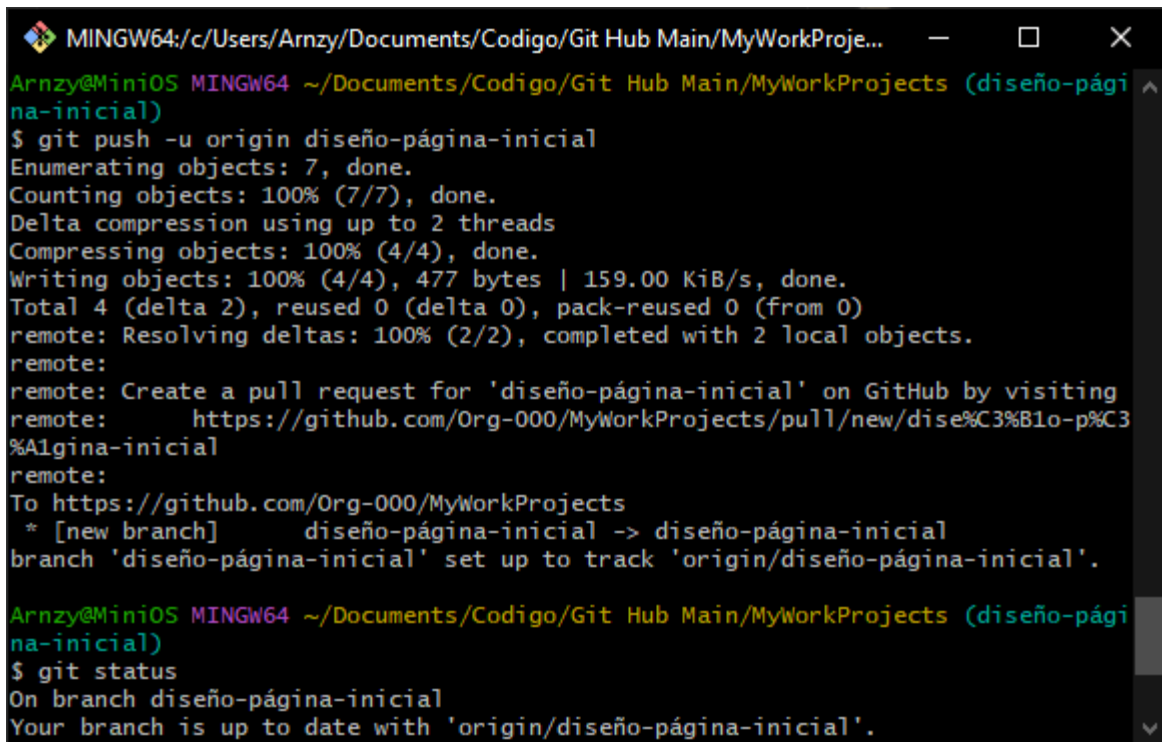
    git push --set-upstream origin diseño-página-inicial

To have this happen automatically for branches without a tracking upstream, see 'push.autoSetupRemote' in 'git help config'.

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (diseño-página-inicial)

```

Ilustración 6. Git Bash - Creación del commit



```

MINGW64:/c:/Users/Arnzy/Documents/Codigo/Git Hub Main/MyWorkProje...
Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (diseño-página-inicial)
$ git push -u origin diseño-página-inicial
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 477 bytes | 159.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'diseño-página-inicial' on GitHub by visiting
remote:   https://github.com/Org-000/MyWorkProjects/pull/new/dise%C3%B1o-p%C3%A1gina-inicial
remote:
To https://github.com/Org-000/MyWorkProjects
 * [new branch]      diseño-página-inicial -> diseño-página-inicial
branch 'diseño-página-inicial' set up to track 'origin/diseño-página-inicial'.

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (diseño-página-inicial)
$ git status
On branch diseño-página-inicial
Your branch is up to date with 'origin/diseño-página-inicial'.

```

Ilustración 7. Git Bash - Carga de archivos a la rama en GitHub con push

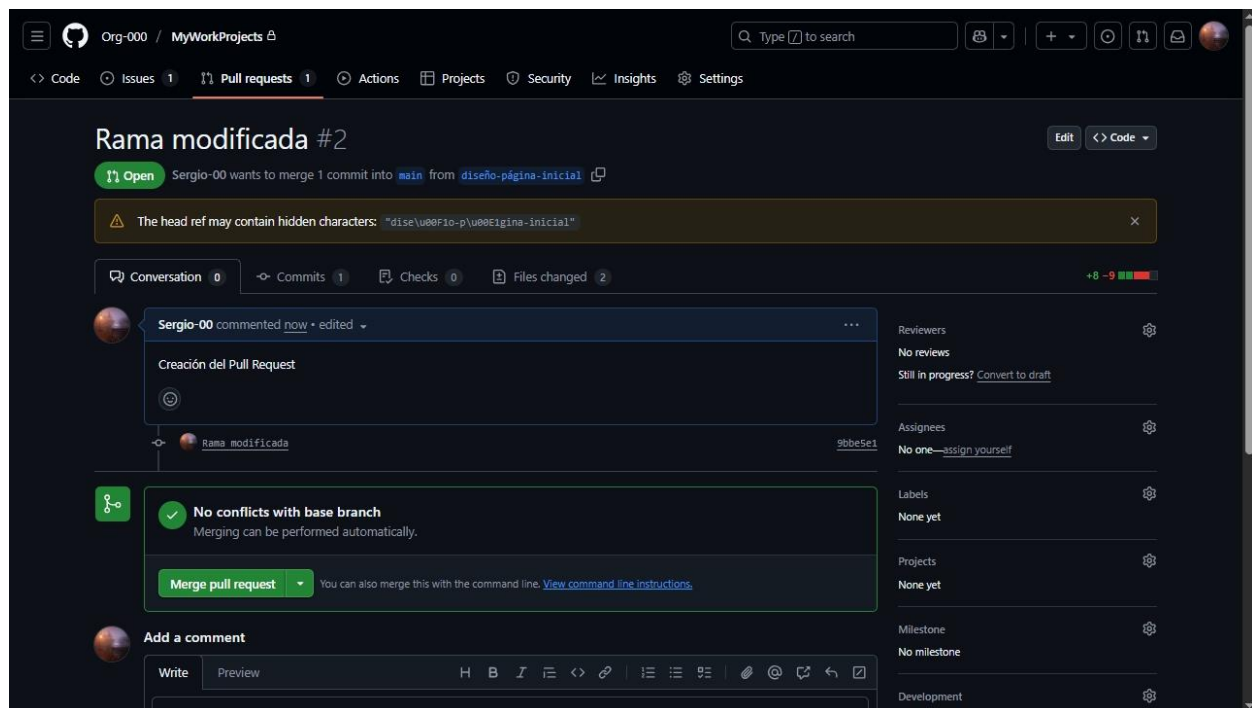


Ilustración 8. GitHub - Creación del pull request

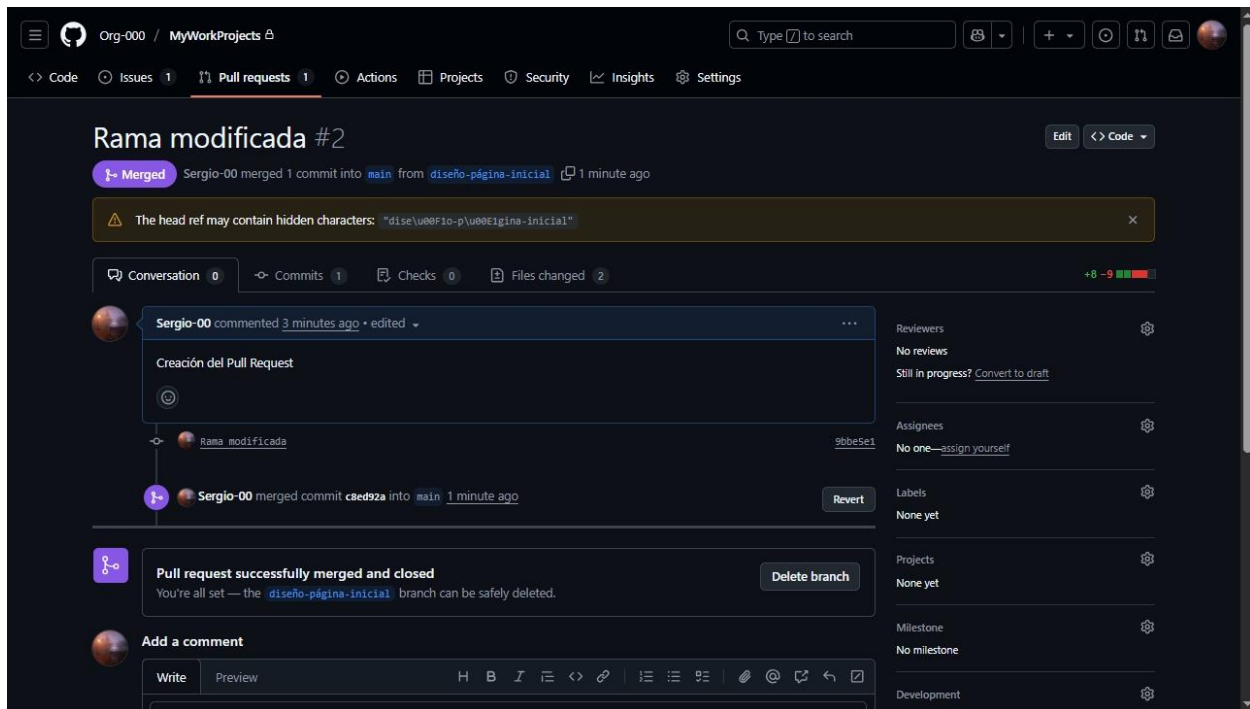


Ilustración 9. GitHub - merge del pull request

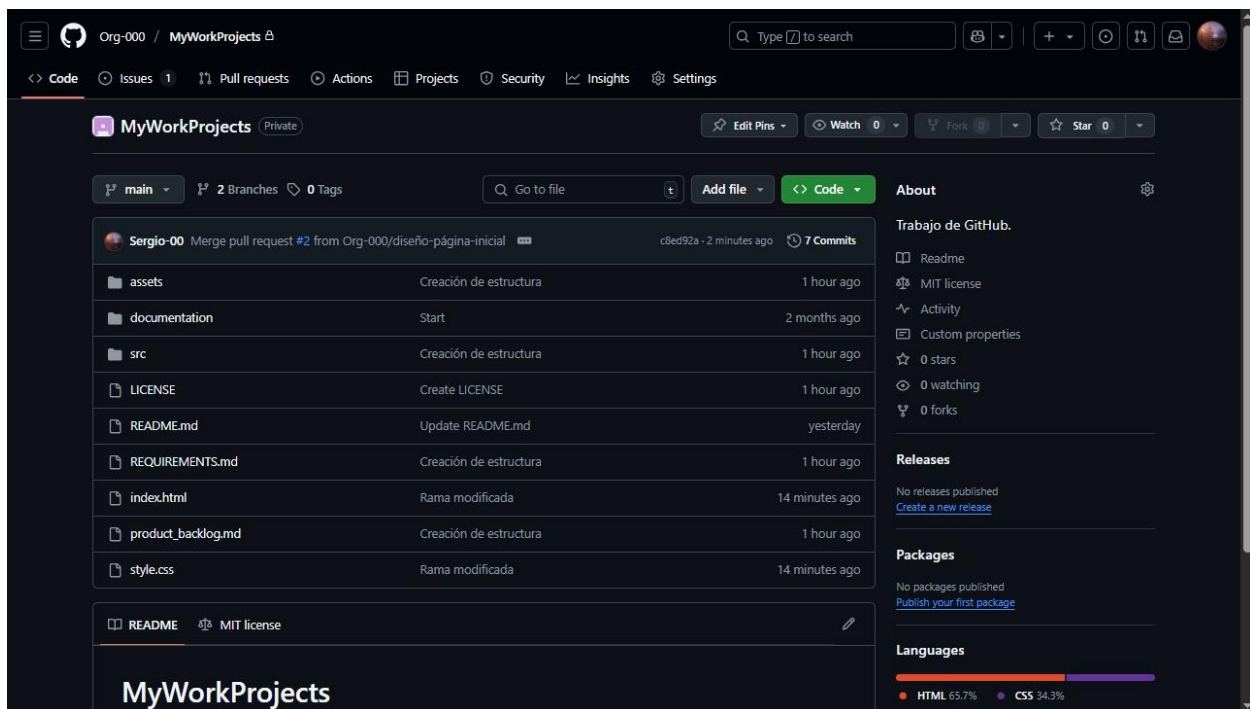


Ilustración 10. GitHub - Repositorio después de la fusión de ramas

Actividad 3: Pull Request e Issues

1. Crear un issue en GitHub para cualquier tarea específica, como "Implementar la evaluación de la rueda de la vida para empleados".
2. Asignar el issue a un miembro del equipo o a usted mismo.
3. Desarrollar la funcionalidad necesaria en una nueva rama (por ejemplo, `evaluación rueda-vida`).
4. Una vez completado, abrir un pull request para fusionar los cambios en la rama principal y mencionar el issue relacionado en el Pull Request.
5. Revisar y discutir el pull request en el equipo, realizar pruebas y hacer ajustes si es necesario.
6. Una vez aprobado, fusionar el pull request y cerrar el issue .

Evidencias:

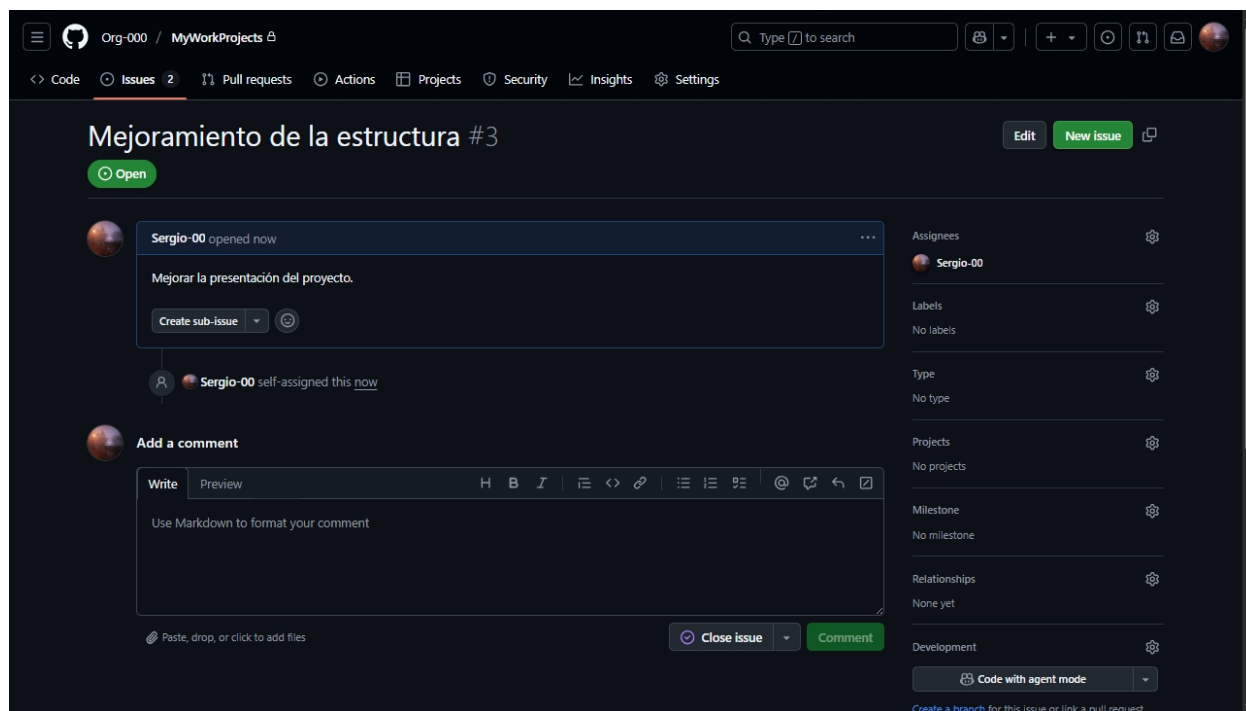
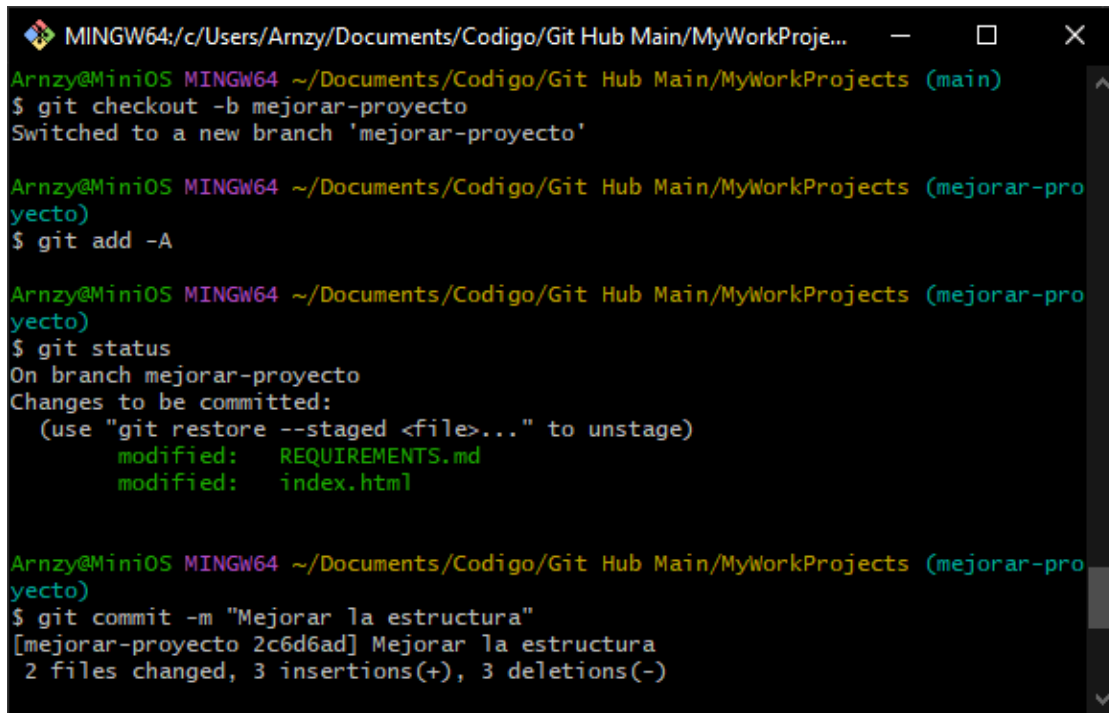


Ilustración 11. GitHub - Creación y asignación del issue



```

MINGW64:/c/Users/Arnzy/Documents/Codigo/Git Hub Main/MyWorkProje...
Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (main)
$ git checkout -b mejorar-proyecto
Switched to a new branch 'mejorar-proyecto'

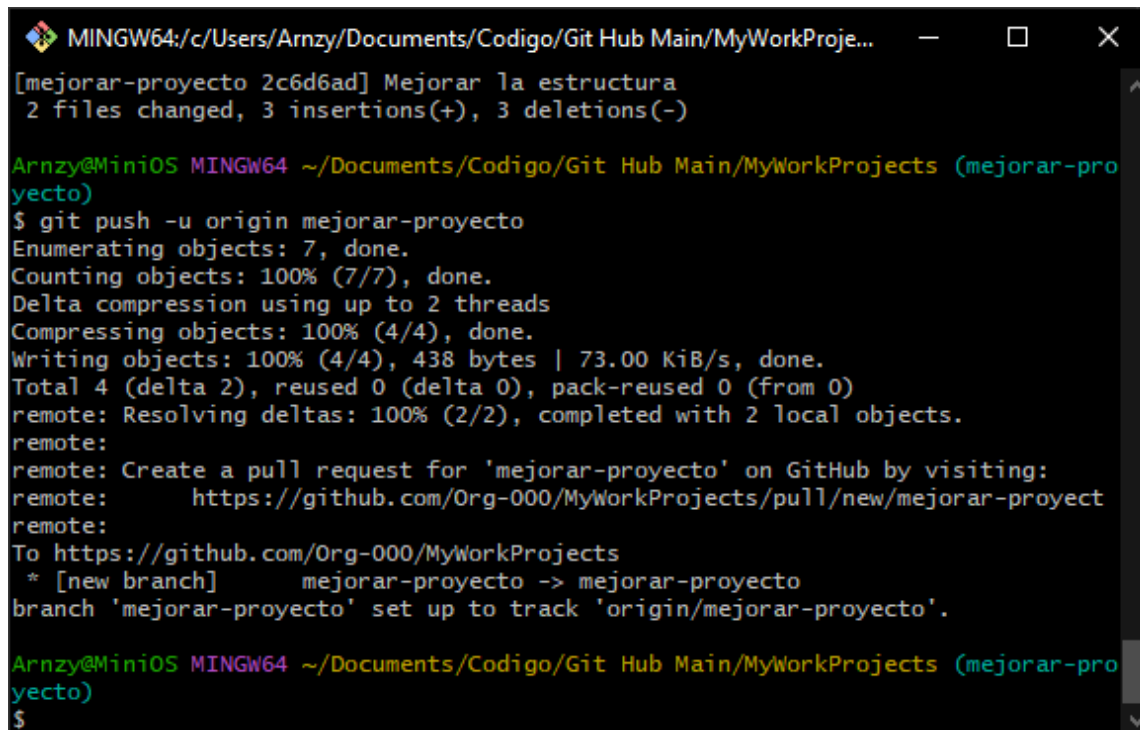
Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (mejorar-proyecto)
$ git add -A

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (mejorar-proyecto)
$ git status
On branch mejorar-proyecto
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   REQUIREMENTS.md
        modified:   index.html

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (mejorar-proyecto)
$ git commit -m "Mejorar la estructura"
[mejorar-proyecto 2c6d6ad] Mejorar la estructura
2 files changed, 3 insertions(+), 3 deletions(-)

```

Ilustración 12. Git Bash - Creación de rama para el issue



```

MINGW64:/c/Users/Arnzy/Documents/Codigo/Git Hub Main/MyWorkProje...
[mejorar-proyecto 2c6d6ad] Mejorar la estructura
2 files changed, 3 insertions(+), 3 deletions(-)

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (mejorar-proyecto)
$ git push -u origin mejorar-proyecto
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 438 bytes | 73.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'mejorar-proyecto' on GitHub by visiting:
remote:   https://github.com/Org-000/MyWorkProjects/pull/new/mejorar-proyect
remote:
To https://github.com/Org-000/MyWorkProjects
 * [new branch]      mejorar-proyecto -> mejorar-proyecto
branch 'mejorar-proyecto' set up to track 'origin/mejorar-proyecto'.

Arnzy@MiniOS MINGW64 ~/Documents/Codigo/Git Hub Main/MyWorkProjects (mejorar-proyecto)
$

```

Ilustración 13. Git Bash - push a la rama para el issue

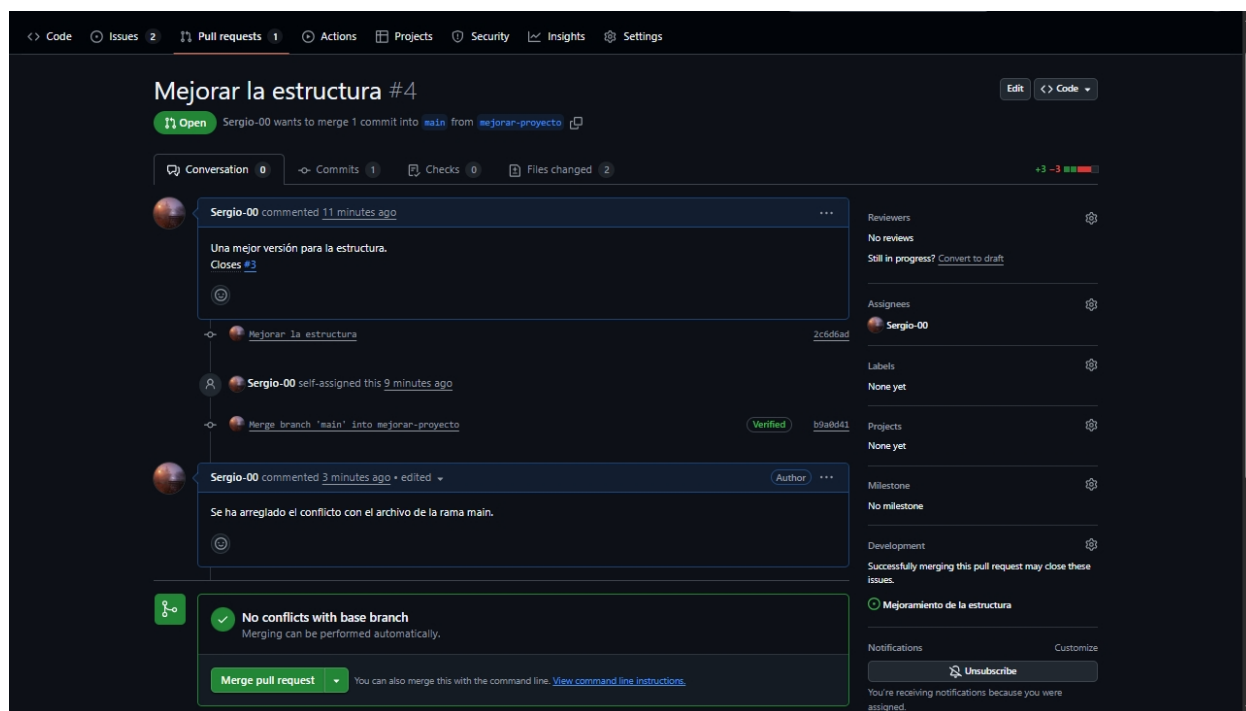


Ilustración 14. GitHub - pull request para el cierre del issue

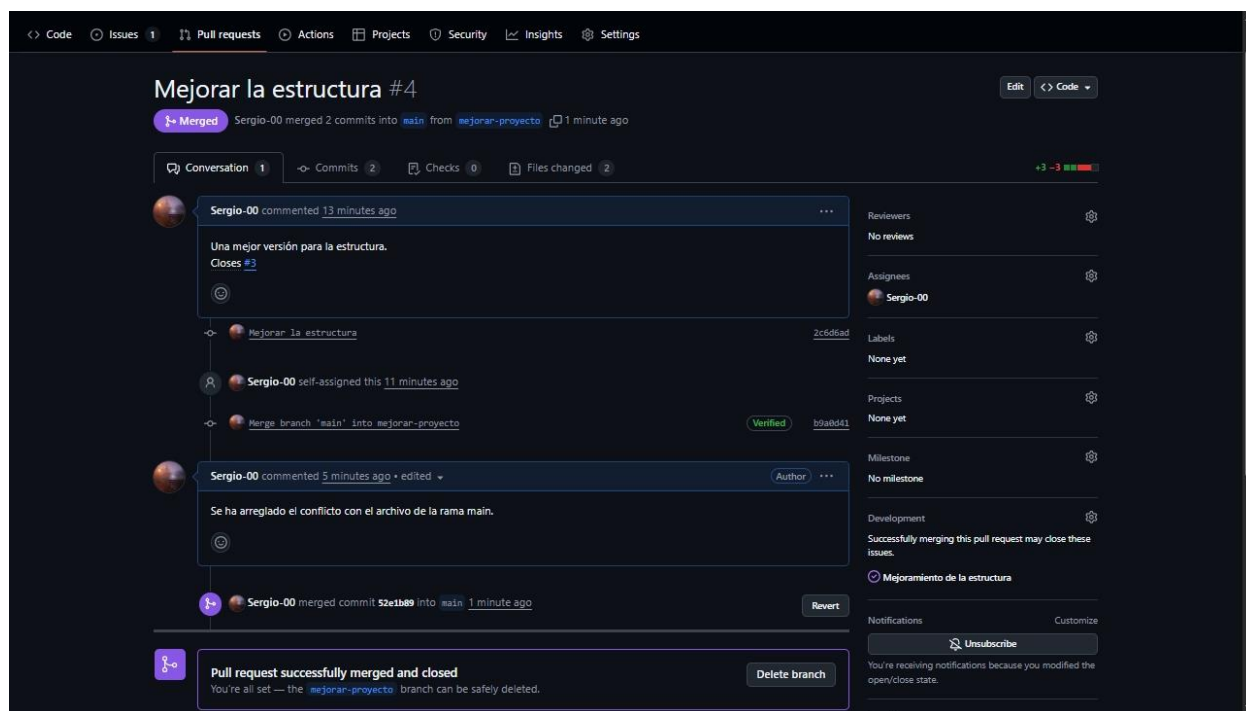


Ilustración 15. GitHub - pull request e issue cerrados

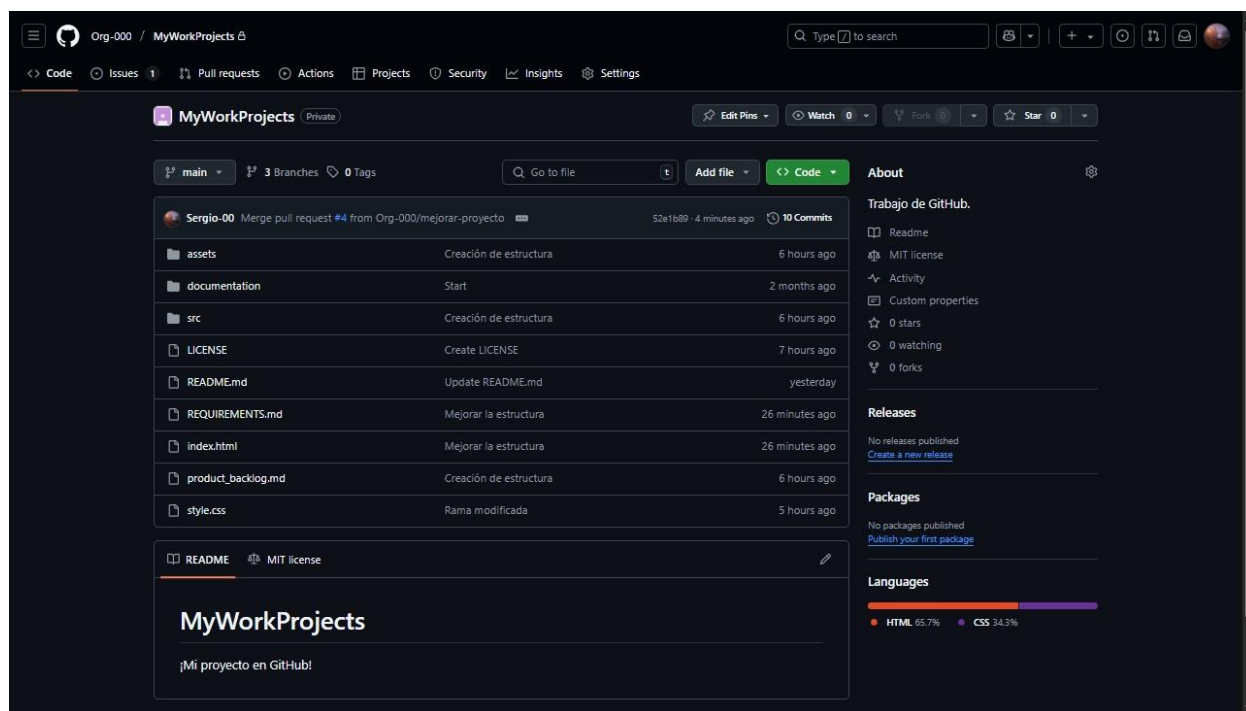


Ilustración 16. GitHub - Repositorio después del cierre del issue

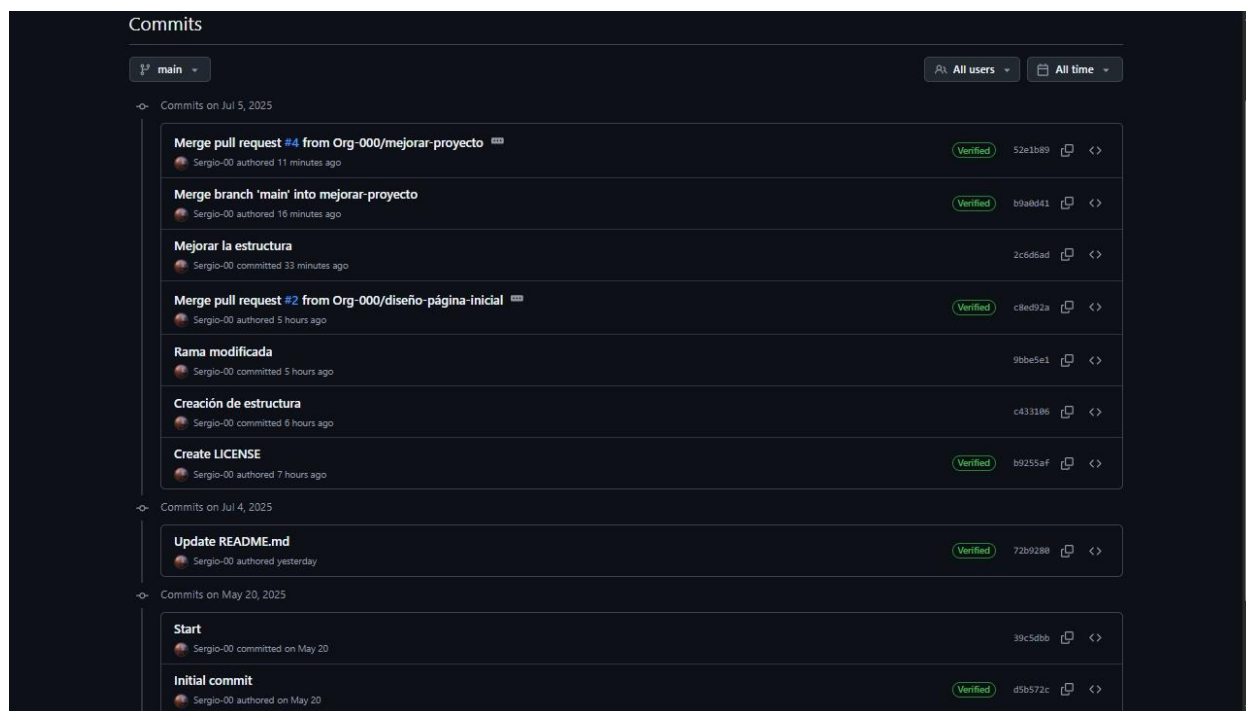


Ilustración 17. GitHub - Historial de cambios

Actividad 4: Actions

1. Configurar un archivo de flujo de trabajo en GitHub Actions para automatizar el proceso de CI/CD (Integración continua y despliegue continuo).
2. Crear un archivo `.yaml` en el directorio `.github/workflows` para definir el flujo de trabajo, por ejemplo, para ejecutar pruebas unitarias sobre el código cada vez que se haga un `push` a la rama principal.
3. Implementar pasos en el flujo de trabajo que automáticamente desplieguen la web en un entorno de producción o staging (como GitHub Pages, Netlify o Heroku) cuando se realice un push.
4. Configurar el flujo de trabajo para ejecutar pruebas de integración y revisar el código en cada pull request, para asegurarse de que no haya errores.
5. Verificar el funcionamiento de GitHub Actions observando los resultados en la pestaña de Actions en GitHub.

Evidencias:

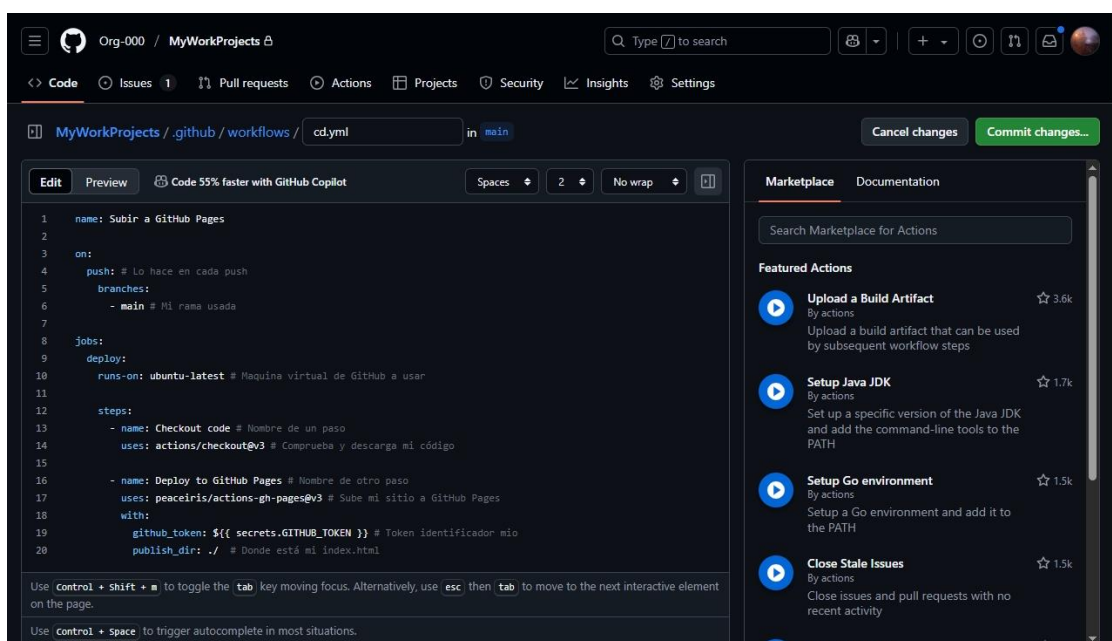


Ilustración 18. GitHub - Archivo de extensión `.yaml` para subir HTML a GitHub Pages

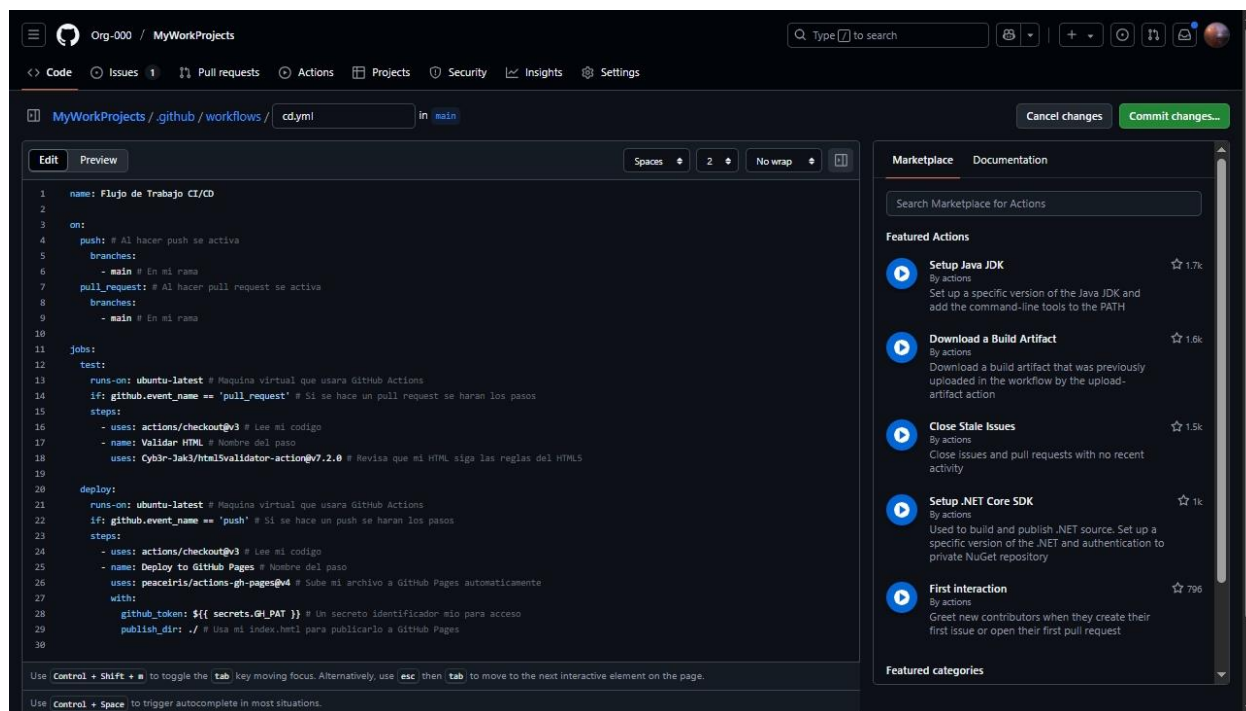


Ilustración 19. GitHub - Cambio del .yml que verifica el HTML con pull request y lo sube a GitHub Pages con push

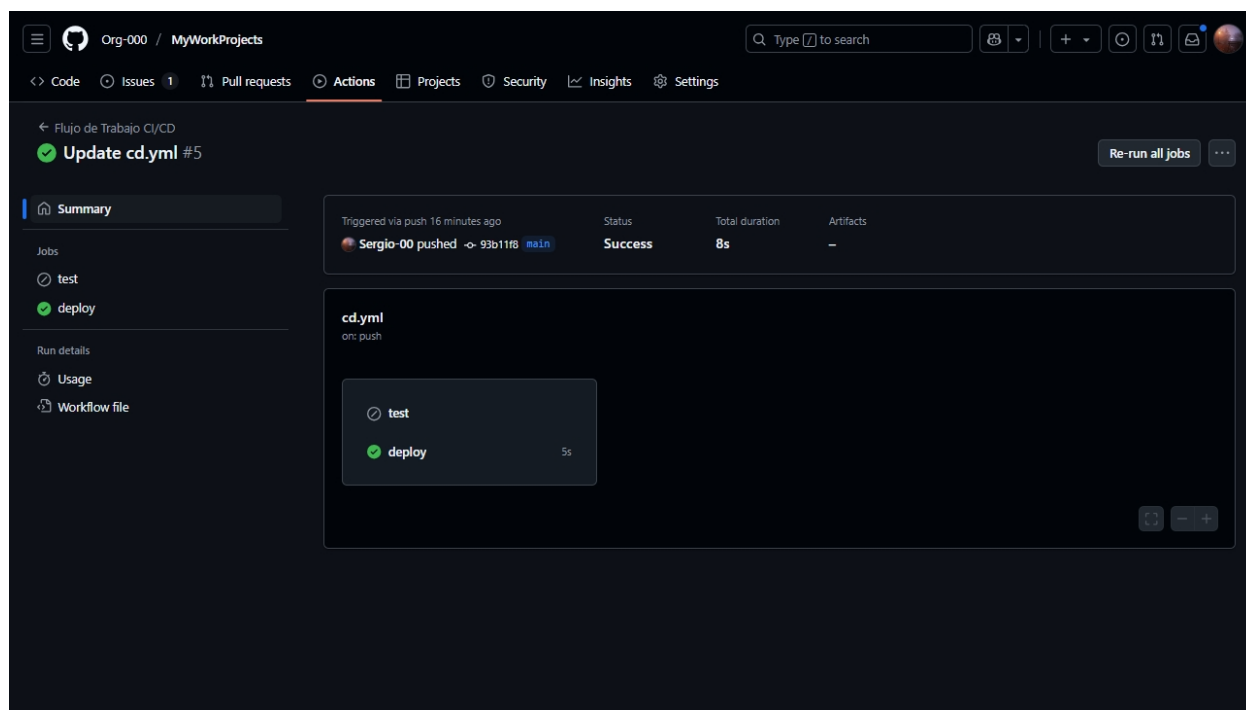


Ilustración 20. GitHub - Demostración de actions ignorando comprobación del HTML, pero subiendo el sitio por un push

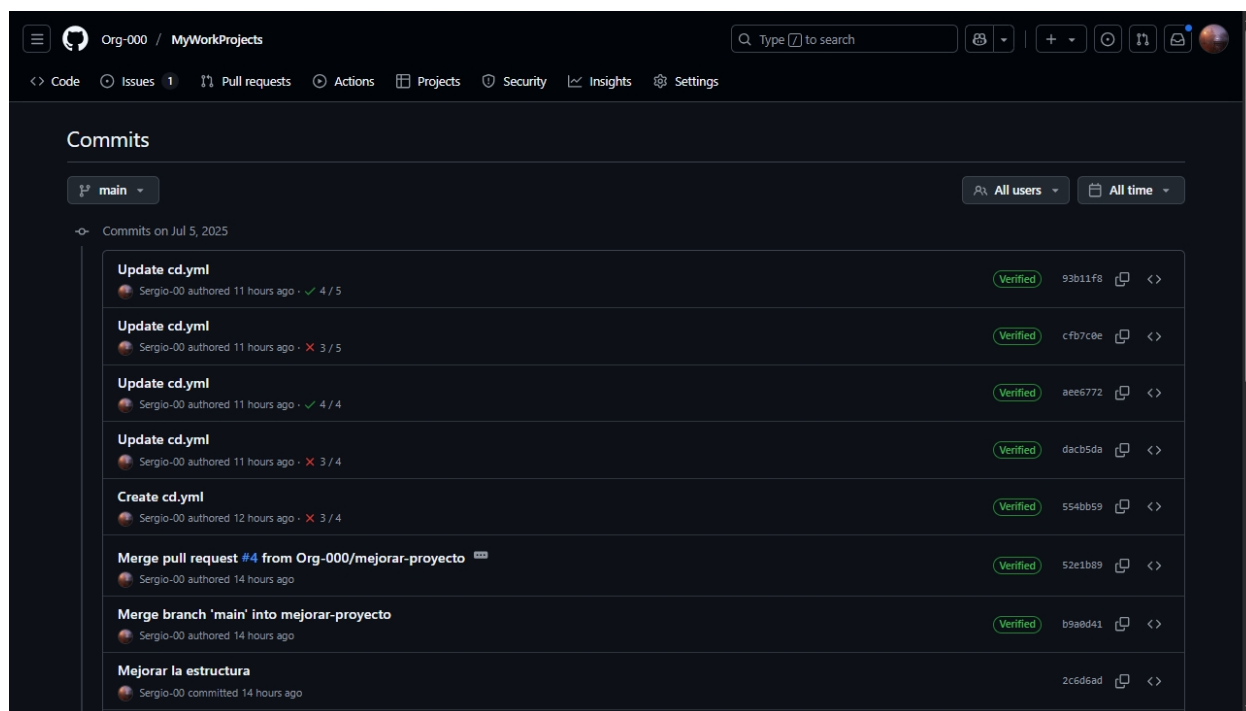


Ilustración 21. GitHub - Historial de cambios del repositorio con el workflow

Bienvenido a Mi Proyecto

Esto es un ejemplo básico de la estructura de un HTML.

Ilustración 22. GitHub Pages - Mi sitio web de prueba <https://org-000.github.io/MyWorkProjects/>

Conclusiones

En conclusión, se ha demostrado como es el procedimiento para usar todas las funciones de GitHub más comunes, descubriendo características útiles para distintas ocasiones que nos sirven para mejorar el orden del código y tener una mayor seguridad de que al hacer código no se creen conflictos, con herramientas como el Git Bash para hacer commits y cargar los cambios hechos al repositorio, los GitHub Actions para automatizar ciertos procesos y el uso de ramas para separar el código en partes manejables.

Referencias Bibliográficas

Casero, A. (2024, 15 de marzo). *Trabajo colaborativo con git: consejos prácticos*. KeepCoding.

<https://keepcoding.io/blog/trabajo-colaborativo-con-git/>

Casero, A. (2025, 2 de enero). *Ramas en git: ¿qué son y cómo usarlas?* KeepCoding.

<https://keepcoding.io/desarrollo-web/que-son-las-ramas-en-git/>

Chuck's Academy. (s.f.). *Pull requests y revisiones de código*. Chuck's Academy.

<https://www.chucksacademy.com/es/topic/git-github/pull-requests-y-revisiones-de-codigo>

Desarrolloweb.com. (2021, 10 de septiembre). *Fork en git*. Desarrolloweb.com.

<https://desarrolloweb.com/articulos/fork-git>

GitHub. (s.f.). *Acerca de los repositorios*. GitHub.

<https://docs.github.com/es/repositories/creating-and-managing-repositories/about-repositories>

Guías Open Source. (s.f.). *El Impacto de los sistemas de control de versiones en la gestión de proyectos de software*. Guías Open Source.

<https://guiasopensource.net/herramientas-de-desarrollo/impacto-sistemas-control-versiones-gestion-proyectos-software/>

Khan. A. B. (2022, 6 de febrero). *Git Commit vs Push*. DelftStack.

<https://www.delftstack.com/es/howto/git/git-commit-vs-push/>